DB First with EF Core

Overview



Introduction to DB First with EF Core

How to use DB First Approach?

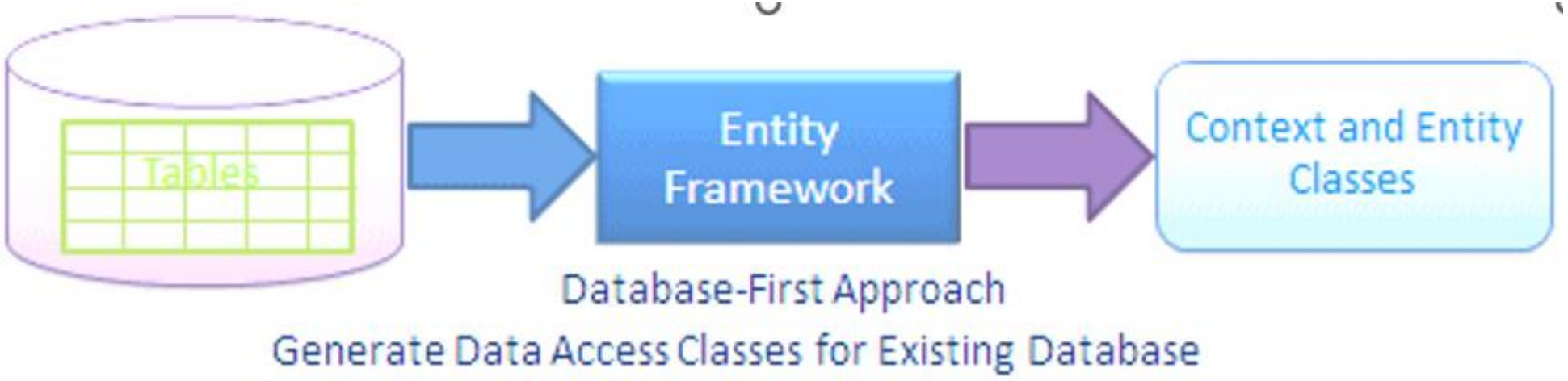
Roles of DbContext in EF Core

Introduction to DB First

Creating a Model for an Existing Database in Entity Framework Core

- Creating entity & context classes for an existing database is called Database-First approach.
- EF Core does not support visual designer for DB model and wizard to create the entity and context classes similar to EF 6.
- So, we need to do reverse engineering using the **Scaffold-DbContext** command.
- This reverse engineering command creates entity and context classes (by deriving DbContext) based on the schema of the existing database.

Creating a Model for an Existing Database in Entity Framework Core



How to use DB First Approach?

- Entity Framework Core can be used with .NET Core or .NET 4.6 based applications. Here, you will learn to install and use Entity Framework Core 2.0 in .NET Core applications using Visual Studio 2017.
- EF Core is not a part of .NET Core and standard .NET framework. It is available as a NuGet package.
- So, we need to do reverse engineering using the Scaffold-DbContext command.
- You need to install NuGet packages for the following two things to use EF Core in your application:
  1. EF Core DB provider
  2. EF Core tools

- **To Install EF Core DB Provider**

- There are different EF Core DB providers available for the different databases. These providers are available as NuGet packages.
- we need to install the NuGet package for the provider of the database we want to access.
- we want to access MS SQL Server database, so we need to install ***Microsoft.EntityFrameworkCore.SqlServer*** NuGet package.

- To Install Package using.**NET Core CLI**

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

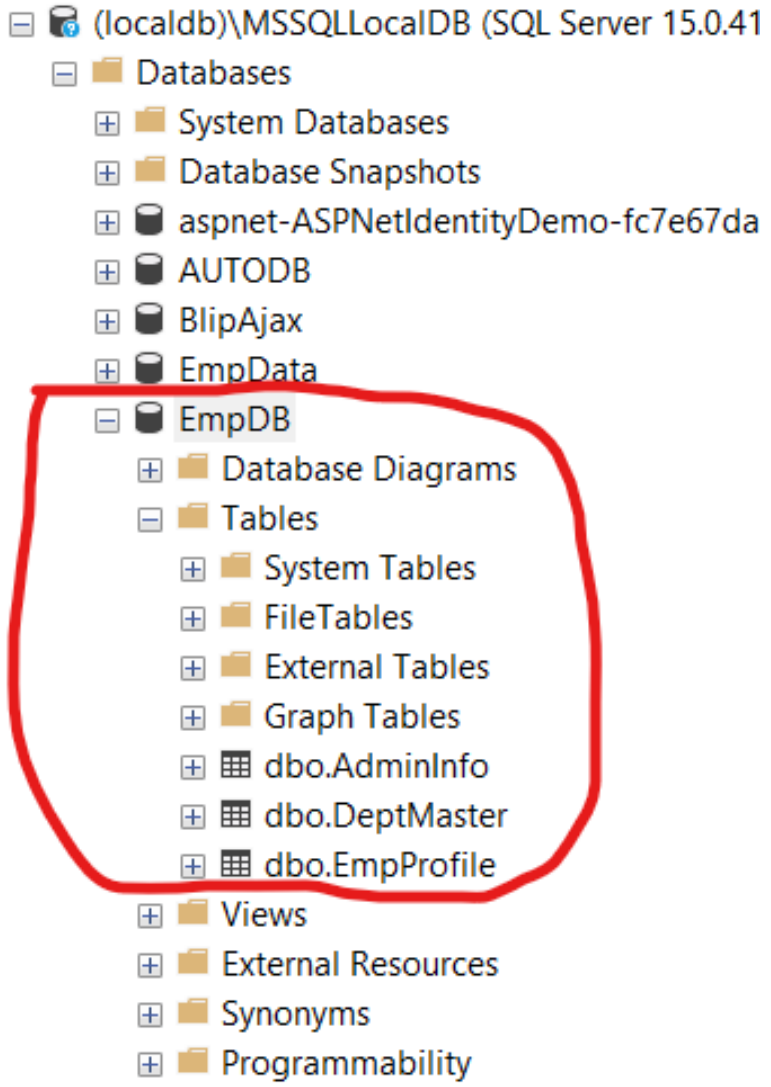
• To Install EF Core Tools

- Along with the DB provider package, you also need to install EF tools to execute EF Core commands.
- These make it easier to perform several EF Core-related tasks in your project at design time, such as migrations, scaffolding, etc.
- In order to execute EF Core commands from Package Manager Console, search for the ***Microsoft.EntityFrameworkCore.Tools*** package from NuGet

- To install pakage using.**NET Core CLI**

```
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

- **Select Existing MS-SQLServer Database**



- **Scaffold-DbContext Command**

Use Scaffold-DbContext to create a model based on your existing database. The following parameters can be specified with Scaffold-DbContext in Package Manager Console:

- **Scaffold-DbContext** [-Connection] [-Provider] [-OutputDir] [-Context] [-Schemas>] [-Tables>] [-DataAnnotations] [-Force] [-Project] [-StartupProject] [<CommonParameters>]

• To Create Project and Execute Scaffold-DbContext Command

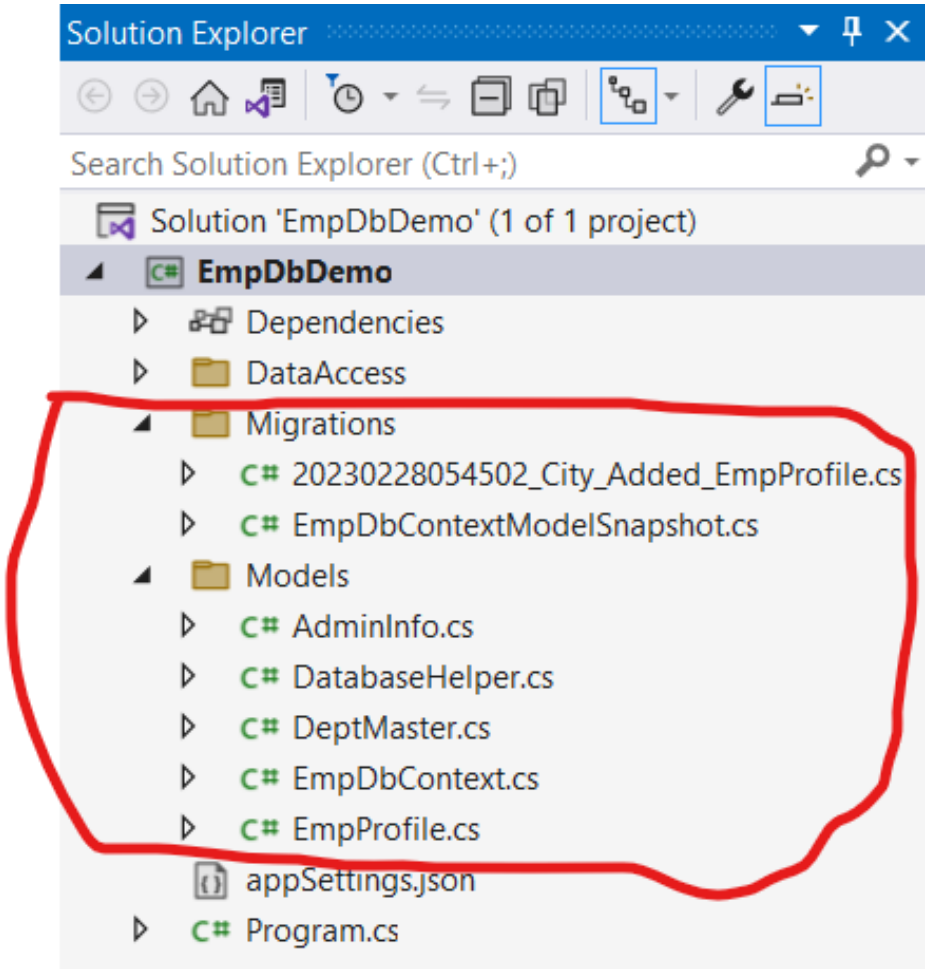
- Open a Visual Studio and create a new project using Console Application (EmpDbDemo).
- In Visual Studio, select menu Tools -> NuGet Package Manger -> Package Manger Console and run the following command:

```
PM> Scaffold-DbContext
"Server=(localdb)\MSSqlLocalDb;Database=EmpDB;Trusted_Connection=True;"
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models
```

- *This will create domain classes inside Models folder as we have mentioned in the above migration command.*

- **Notice the Changes made in the current Project**

- Open a Visual Studio solution Explorer and see the Models and Migration Folder.



What is DbContext in EF Core

What is DbContext class?

The DbContext class is a fundamental component in Entity Framework (EF) and EF Core. It serves as the main entry point for interacting with the database, managing entities, and performing database operations.

The DbContext represents a session or a unit of work that encapsulates the database connection and provides a set of APIs to query, insert, update, and delete data.

```
using Microsoft.EntityFrameworkCore;
namespace EmpDbDemo.Models;

public partial class EmpDbContext : DbContext
{
    public EmpDbContext()
    {
    }

    public EmpDbContext(DbContextOptions<EmpDbContext> options)
        : base(options)
    {
    }
    public virtual DbSet<AdminInfo> AdminInfos { get; set; }

    public virtual DbSet<DeptMaster> DeptMasters { get; set; }

    public virtual DbSet<EmpProfile> EmpProfiles { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        if (!optionsBuilder.IsConfigured)
        {
            optionsBuilder.UseSqlServer(DatabaseHelper.GetConnectionString());
        }
    }
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<AdminInfo>(entity =>
        {
            entity.HasKey(e => e.Email).HasName("PK__AdminInf__A9D10535061D9C61");

            entity.ToTable("AdminInfo");

            entity.Property(e => e.Email).HasMaxLength(100);
            entity.Property(e => e.Pass).HasMaxLength(20);
        });

        modelBuilder.Entity<DeptMaster>(entity =>
        {
            entity.HasKey(e => e.DeptCode).HasName("PK__DeptMast__BB9B955111A40393");

            entity.ToTable("DeptMaster");

            entity.Property(e => e.DeptCode).ValueGeneratedNever();
            entity.Property(e => e.DeptName).HasMaxLength(50);
        });
```

```
modelBuilder.Entity<EmpProfile>(entity =>
      {
         entity.HasKey(e => e.EmpCode).HasName("PK__EmpProfi__7DA847CB618BAF9A");

         entity.ToTable("EmpProfile");

         entity.HasIndex(e => e.Email, "UQ__EmpProfi__A9D105347A666157").IsUnique();

         entity.Property(e => e.EmpCode).ValueGeneratedNever();

         entity.Property(e => e.City).HasMaxLength(50);

         entity.Property(e => e.DateOfBirth).HasColumnType("datetime");

         entity.Property(e => e.Email).HasMaxLength(100);

         entity.Property(e => e.EmpName).HasMaxLength(50);

         entity.Property(e => e.PhoneNo).HasColumnType("numeric(10, 0)");

         entity.HasOne(d => d.DeptCodeNavigation).WithMany(p => p.EmpProfiles)
            .HasForeignKey(d => d.DeptCode)
            .HasConstraintName("FK__EmpProfil__DeptC__276EDEB3");
      });

      OnModelCreatingPartial(modelBuilder);
   }
}
```

Roles of DbContext in EF Core

Roles of DbContext class

Manage database connection:

The DbContext establishes a connection to the database using a database provider such as SQL Server, MySQL, or SQLite. It manages the connection lifecycle, including opening and closing the connection, and provides access to the underlying database provider.

Entity Sets (DbSet): The DbContext exposes DbSet properties representing the entity sets (database tables or views) in the database. Each DbSet represents a collection of entities of a particular entity type. These sets allow querying, inserting, updating, and deleting entities in the corresponding database tables.

Entity Configuration: The DbContext contains configuration and mapping information that defines how the entity classes are mapped to the database tables. This configuration can be done using attributes, fluent API, or conventions, allowing customization of the entity-to-table mappings, relationships, and other database-specific settings.

Change Tracking: The DbContext tracks changes made to entities within its scope. It keeps track of entities that have been modified, added, or deleted and generates appropriate SQL statements to persist these changes to the database when SaveChanges method is called.

Querying: The DbContext provides query capabilities through LINQ (Language Integrated Query) and a query API. Developers can query entities using LINQ syntax, which gets translated into SQL queries by the underlying database provider. The DbContext executes the queries and returns the results as entity objects.

Database Operations: The DbContext enables database operations such as inserting, updating, and deleting entities. It exposes methods like Add, Update, and Remove to perform these operations, and the changes are persisted to the database when SaveChanges is called.

Performance Optimization: The DbContext allows configuring caching, lazy loading, and other performance optimization techniques to improve the efficiency of data access and minimize the number of database round trips.

In summary, the DbContext class acts as a mediator between the application code and the database. It provides a high-level abstraction for managing entities, database connections, and database operations, making it easier to work with data persistence in EF and EF Core applications.

References

https://www.entityframeworktutorial.net/efcore/create-model-for-existing-database-in-ef-core.aspx

https://www.c-sharpcorner.com/article/entity-framework-database-first-in-asp-net-core2/

https://www.entityframeworktutorial.net/efcore/entity-framework-core.aspx

https://www.c-sharpcorner.com/article/entity-framework-database-first-in-asp-net-core2/

https://www.c-sharpcorner.com/article/entity-framework-core-6-with-database-first/

https://docs.devart.com/dotconnect/postgresql/EFCore-Database-First-NET-Core.html

https://dotnettutorials.net/lesson/entity-framework-database-first-approach/